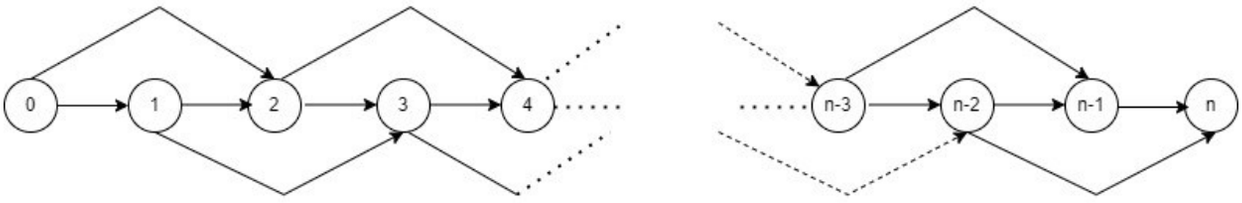
Name: _____ ID: _____

CSE 2046 / CSE 2246 Final Exam

11.06.2024 – 105 minutes

Q1	Q2	Q3	Q4	Q5	SUM
/20	/20	/20	/35	/25	/100

Q-1 (20 pts) *Dynamic Programming.* Consider a graph with n+1 nodes as given in the following figure.



A robot starts at node 0 and aims to reach node *n*. At each node, it can move to the next node or skip one and proceed to the node after that. Each node *i* has an associated positive cost *C[i]* for the robot, except for nodes 0 and n, where the cost is zero. (i.e. *C*[0] = *C*[*n*] = 0). **The goal is to move from node 0 to node *n* with minimum total cost.**

For example, if C = [0, 10, 15, 20, 0], then the robot will visit node 0, 2 and 4, and the minimum total cost would be 15.

If C = [0, 1, 1, 10, 1, 10, 1, 1, 10, 0], then the robot will visit nodes 0, 2, 4, 6, 7, and 9, and the minimum total cost would be 4.

Describe a **dynamic programming algorithm** for this problem. Apply your algorithm for the following instance: C = [0, 10, 5, 20, 15, 25, 30, 0]

Hint: Define f[*i*] as the minimum total cost from node *i* to node *n*. Set up a recurrence relation for f[*i*].

$f[i] = cost[i] + \min(f[i+1], f[i+2])$

$f[n-1] = cost[n-1]$

$f[n-2] = cost[n-2]$.

Recurrence relation

```
for (i = n-3; i >= 0; i--)
    f[i] = cost[i] + min(f[i+1], f[i+2])

return (f[0])
```

Application:

$f[6]=30 \quad f[5]=25$

$f[4] = 15 + \min\{25, 30\} = 40$

$f[3] = 20 + \min\{40, 25\} = 45$

$f[2] = 5 + \min\{45, 40\} = 45$

$f[1] = 10 + \min\{45, 45\} = 55$

$f[0] = 0 + \min\{55, 45\} = 45$

Then we can go backwards, to determine visited nodes. $0 \to 2 \to 4 \to 5 \to 7$

Alternative:

```
f[0] = cost[0]
f[1] = cost[1]
for (i = 2, i <= n, i++)
    f[i] = min(f[i-1] + cost[i], f[i-2] + cost[i]);
return (f[n])
```

here f[i] is defined as cost of climbing from bottom to i.

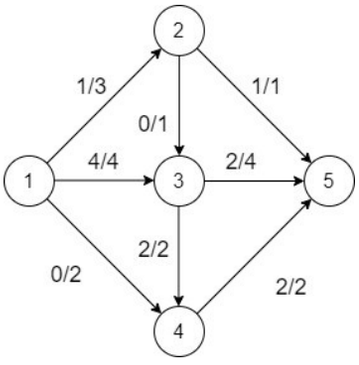
Application: $f[0]=0, f[1]=10, f[2]=5+\min\{0,10\}=5, f[3]=20+\min\{10,5\}=25, f[4]=15+\min\{5,20\}=20$

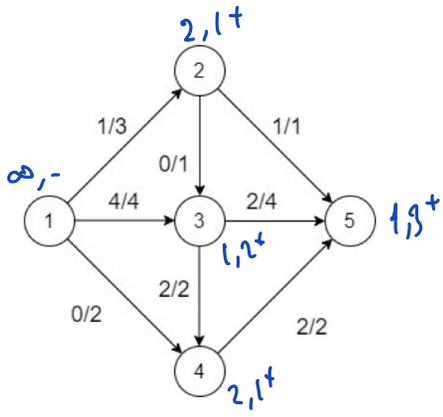
$f[5]=25+\min\{25,20\}=45, f[6]=30+\min\{20,45\}=50 \quad f[7]=0+\min\{45,50\}=45$

Name: _____ ID: _____

Q-2 (20 pts). *Maximum flow.* Consider the following network where 1 is the source and 5 is the sink. Some of the flows are already assigned as shown in the figure.

(a – 12 pts) Continue from the given step and apply the shortest augmenting path algorithm to find the maximum flow. Show the BFS queue and all the labels in every step.

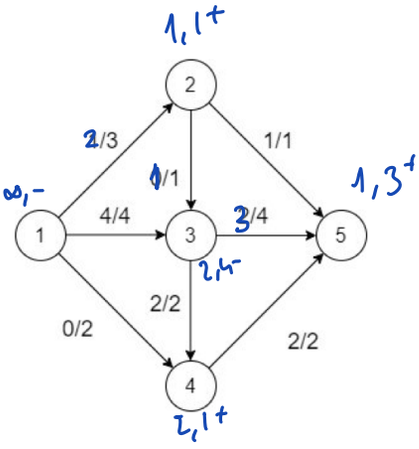




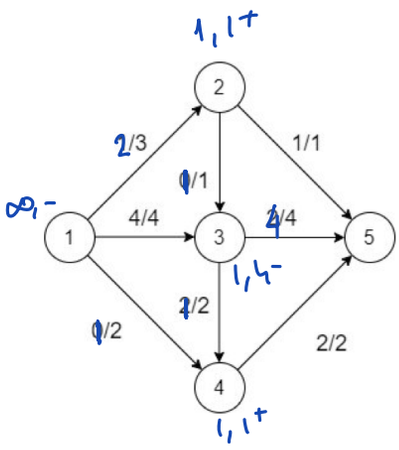
Q: 1 2 4 3 5

aug. path: 1→2→3→5

flow: 1



Q: 1 2 4 3 5



Q: 1 2 4 3

(b – 3 pts) What is the minimum cut found by the algorithm?

from labeled to unlabeled in the last step: $\{(2,5),(3,5),(4,5)\}$

value of min cut = 7

(c – 5 pts) For any network, describe whether the following arguments are true or false. (Answers without reasoning will not be graded.)

i. If we double the capacities of all the edges included in the minimum cut, then the value of the maximum flow doubles.

False. There could be other bottlenecks.

ii. If we reduce by half the capacities of all the edges included in the minimum cut, then the value of the maximum flow reduces by half.

True. New edges will still be in the min cut, since min cut = max flow, value of max flow will be reduced by half.

Name: _____ ID: _____

Q-3. (20 pts) Suppose that we have a file consisting of large DNA strings. All the possible characters are A, C, G, T or blank character ("_").

It is known that the most frequent character is A with 40% frequency. The least frequent character is "_" with 10% frequency. We are going to apply Huffman coding.

(a – 10 pts) Given only the above information, what is the maximum possible codeword length for the blank character "_"? Prove your answer.

In any Huffman Tree, maximum depth is n-1, so maximum codeword length is also n-1. So, for the given instance, we have to show whether it is possible to have a codeword length of 4. We can show that it is possible (to show any example is ok.)

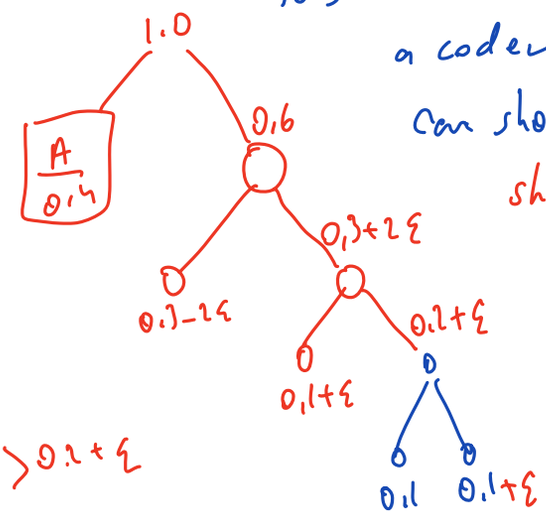
A = 0.4

_ : 0.1

C: 0.1+ε

G: 0.1+ε

T: 0.3−2ε

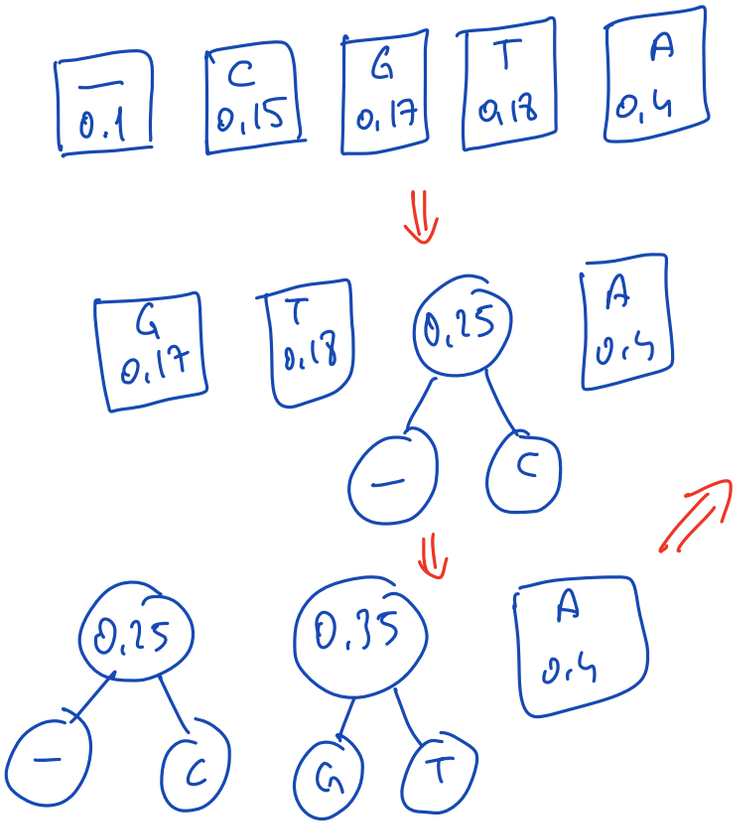


If 0.3−2ε > 0.2+ε

i.e. ε < 0.1/3

∴ max possible codeword length is 4.

(b – 10 pts) Apply Huffman coding for the case where the frequencies of C, G, and T are 15%, 17%, and 18% respectively. Find the compression ratio with respect to the minimum fixed-length encoding.



Avg codeword length = 0.4×1 + 0.6×3 = 2.2

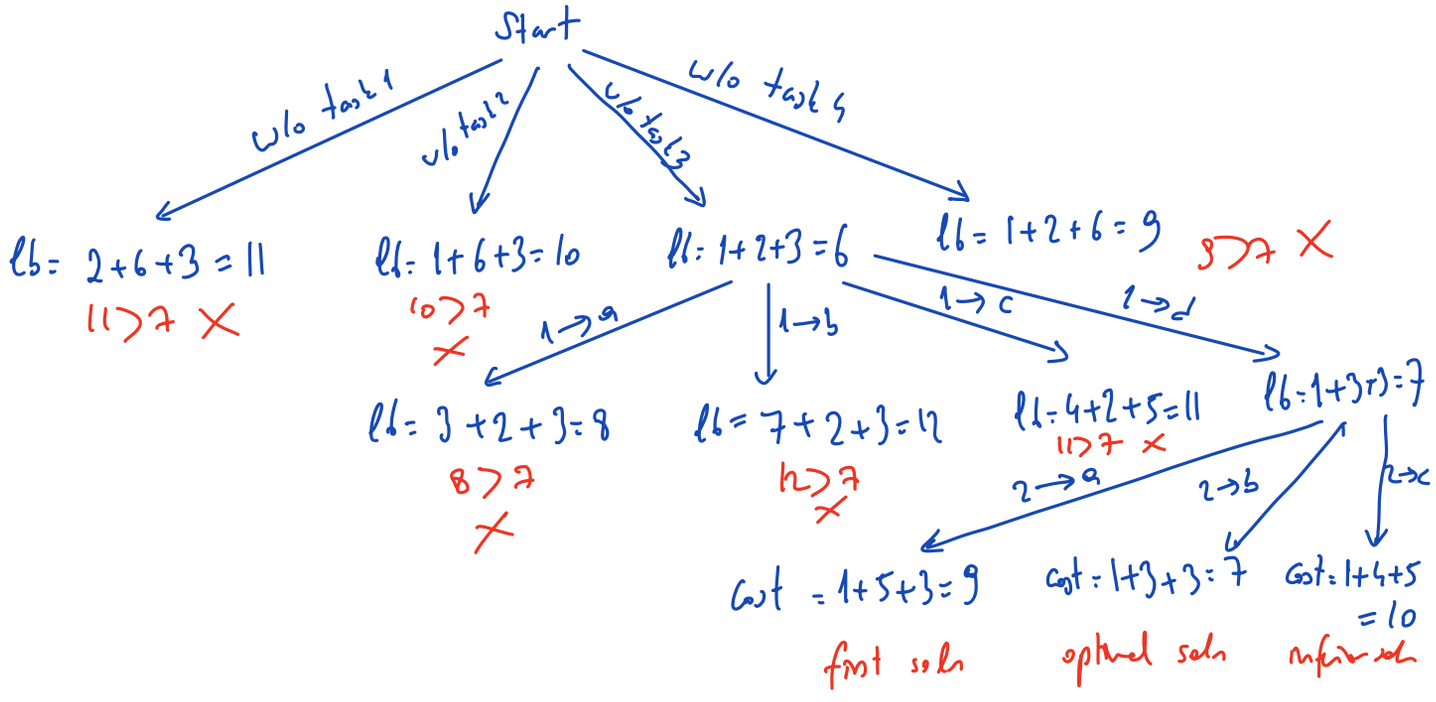
Compression: (3−2.2)/3 = 0.8/3

Name: _____ ID: _____

Q-4. (35 pts) Consider a **modified version of Assignment problem**. There are *n agents* and *m* tasks (where $m \leq n+1$). Any agent can be assigned to perform any task, incurring some *cost* that may vary depending on the agent-task assignment. It is required to perform **ALL TASKS EXCEPT ONE** (it is enough to perform m-1 tasks) by assigning exactly one agent to each task in such a way that **the *total cost* of the assignment is minimized**. Note that, some of the agents may not be assigned to any task and stay idle.

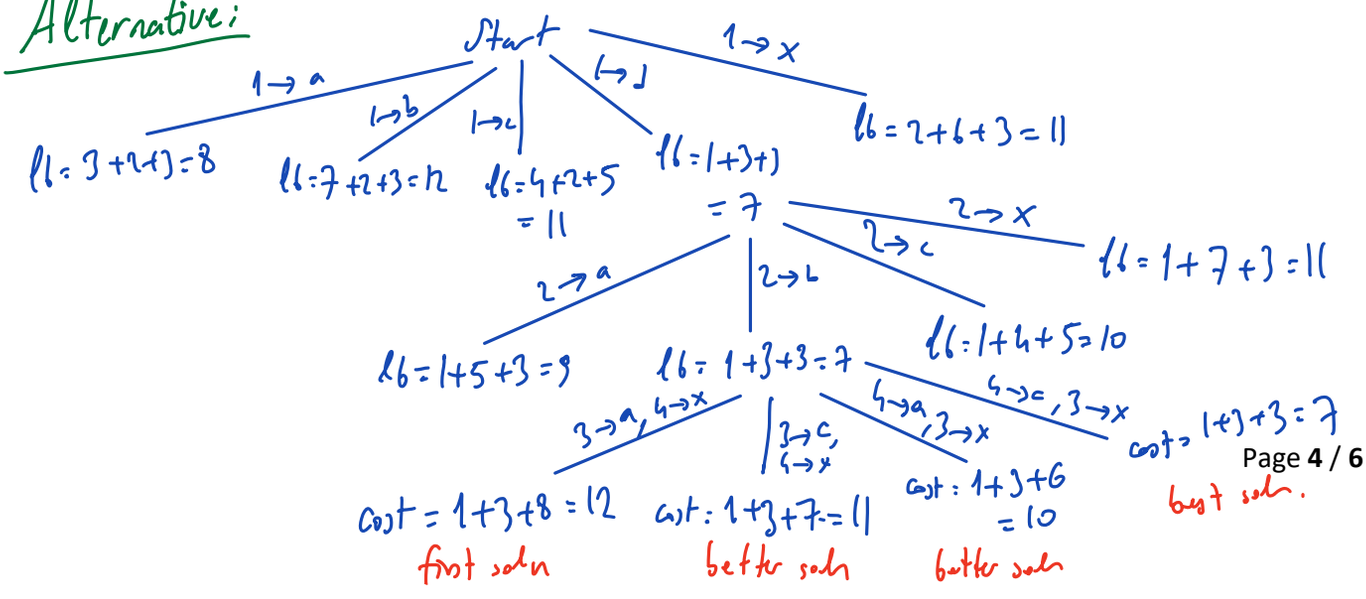
(a – 20 pts) For the following utility matrix, define a bounding function and apply a best-first **branch-and-bound algorithm**.

	Task 1	Task 2	Task 3	Task 4
Agent a	**3**	**5**	**8**	**6**
Agent b	**7**	**3**	**9**	**5**
Agent c	**4**	**4**	**7**	**3**
Agent d	**1**	**2**	**6**	**5**



Lower bound is defined as the sum of smallest value in every column, except one which is decided in the first step.

Alternative:



Name: _____ ID: _____

(b – 15 pts) Describe a greedy algorithm for the above problem (give step-by-step description), and apply it on the same instance (given below). What is the obtained accuracy ratio? What is the maximum possible accuracy ratio? (Give an upper bound on the performance ratio, or prove that it is unbounded above).

	Task 1	Task 2	Task 3	Task 4
Agent a	3	5	8	6
Agent b	7	3	9	5
Agent c	4	4	7	3
Agent d	1	2	6	5

Step 1: Among all cost values, choose the one with minimum value, such that the corresponding task is not assigned to any agent. (make that assignment)

Step 2: Repeat Step 1 if # assigned tasks < n-1.

- Assign agent d to task 1
- Skip agent d - task 2 assignment.
- Assign agent b to task 2
- Assign agent c to task 4.
- Stop. Accuracy ratio is 1, because we've found the optimal.

In the above example, if agent a - task 3, agent a - task 4, agent c - task 3, agent c - task 4 assignment costs were arbitrarily high, the accuracy ratio would be arbitrarily high, so the performance ratio is unbounded above!

Q-5. (25 pts) (a – 5 pts) What is the trivial lower bound of the problem given in Q-1? Is this lower bound tight? Why?

Trivial LB of the problem given in Q-1 is $\Omega(n)$ because we have to process all the values in C[] array (with size n).

This LB is tight, because time complexity of DP algorithm is $O(n)$.

Name: _____ ID: _____

(b – 6 pts) Prove that the modified version of the assignment problem given in Q-4 is at least as hard as the ordinary assignment problem.

To show this, we reduce ordinary assignment problem to the modified version. Take the input matrix. Add a new task with very high cost values for each agent. Run the algorithm for modified version, return the assignment. Since ordinary assignment can be reduced to modified version in linear time, modified version is at least as hard as the ordinary assignment problem.

(c – 8 pts) Suppose that the element uniqueness problem can be reduced to Problem X, and Problem X can be reduced to CNF-Satisfiability problem (which is known to be NP-Complete). All the reductions can be done in linear time. For each of the following, describe whether it is true, false or unknown, with sufficient reasoning.

i. Problem X is NP-Complete. Unknown.

We can't say anything. If we reduce an NP-Complete problem to X, we may argue that problem X is also NP-Complete, but reducing X to an NP-Complete problem proves nothing.

ii. Problem X can be reduced to the Traveling Salesman Problem in polynomial time.

True. Because Problem X can be reduced to CNF-SAT, and NP-Complete problems can be reduced to each other.

(d – 6 pts) Give an example problem for each of the following:

i. A tractable combinatorial problem

MST problem, Shortest path

ii. An intractable decision problem

Turing's Halting Problem

iii. A problem that we do not know whether it is tractable or not.

TSP, Knapsack